TiDPI 07 - Rzetelny benchmark C++ i Python

Kryptonim TiDPI: Regulus

2026-10-17

Sprawozdanie TiDPI 07 - rzetelny benchmark C++ i Python

1. Kryptonim TiDPI i wykonany poziom

Kryptonim: **Regulus**. Wykonany zakres: **A**, dnia 2026-10-17. Jedna seria, dwa bloki tej samej konfiguracji. Poziomów B-D nie realizowano. Operacja podana jako uzgodniona z prowadzącym: wielokrotne obliczanie sumy odległości liczb od wartości 500.

Źródłem wymagań jest cały lokalny materiał TiDPI_07 (10 stron): rdzeń A i pliki na s. 6, rozszerzenia B-D na s. 6-8, sprawozdanie i kryteria na s. 8-10. Ogólna lista na s. 9 obejmuje również artefakty wyższych poziomów; przypadek błędny, druga pełna seria i CSV porównawczy należą do B. Nie deklaruje się tych elementów jako wykonanych. Plan przed pomiarami, niezależny wzorzec, rotacja bloków, TC i szczegółowe dowody wynikają dodatkowo z polecenia użytkownika. Lokalny `AGENTS.md` był pusty i nie ustanawiał dodatkowych wymagań ani raportów pomocniczych.

Z raportów TiDPI_03, TiDPI_05 i TiDPI_06 wykorzystano organizację sekcji, rejestr dowodów, metryczkę i jawny opis ograniczeń. Nie przenoszono ich wyników. Szkolnego PDF nie dołącza się do rozwiązania. Nie użyto ilustracji, więc katalog `images/` nie jest potrzebny. Eksport: raport PDF.

Pliki rozwiązania

- C++, Python, generator.
- Wspólne dane, plan sprzed pomiarów.
- Zapis jednej serii A, kontrola danych i rekordów.
- Wyniki C++, wyniki Python.
- Eksport raportu; odtworzenie: `sh export_report.sh` (Pandoc i XeLaTeX).
- Polecenia serii.

2. Cel i hipoteza

Pytanie: jakie czasy osiągną dwie ręczne implementacje tej samej sumy odległości, gdy dane, liczba rund i granice pomiaru są wspólne? Hipoteza zapisana w D01: obie implementacje zwrócą dokładnie wynik wzorcowy; czasy mogą się różnić, bez wybierania zwycięzcy przed pomiarem. Celem jest poprawny rdzeń benchmarku, a nie ustalenie uniwersalnej szybkości języków.

Niezależnie oczekiwany wynik: **1 002 000 000** po **200 rundach**. To wynik obliczeń; odcisk SHA-256 identyfikuje bajty pliku i ma inne znaczenie.

3. Środowisko oraz konfiguracja

Metryczka D02 została zebrana 2026-10-17 o 10:22:26 czasu Europe/Warsaw. Pomiary wykonano lokalnie, kolejno, bez debuggera.

Element	Faktycznie użyta wartość (D02)
Sprzęt	Apple M4 Pro, arm64, 24 GiB RAM
System	macOS 27.0.1, build 26A434
Kompilator	Apple Clang 21.0.0, clang-2100.3.34.2
Target	arm64-apple-darwin27.0.0
Python	CPython 3.14.4
C++	C++17, Release, `-O2 -DNDEBUG`

Rzeczywiste polecenie budowania (D04), z katalogu TiDPI_07:

```
clang++ -std=c++17 -O2 -DNDEBUG -Wall -Wextra -pedantic \
  benchmark.cpp -o _build/benchmark
```

`-O2` włącza optymalizację; `-DNDEBUG` definiuje makro konfiguracji Release; standardowe `assert` nie są używane do kontroli badanego C++. Kod kompilatora wyniósł 0; oba strumienie kompilacji są puste. Nie używano Windows, Visual Studio ani MSVC. Jest to lokalna adaptacja procedury szkolnej na macOS/arm64/Clang.

C++ używa `steady_clock`, `is_steady=1`, nominalny okres typu zegara 1/1 000 000 000 s. To okres reprezentacji, nie zmierzona efektywna rozdzielczość. Tej ostatniej nie wyznaczono. Python używa `perf_counter_ns()`; `get_clock_info` zgłosił `mach_absolute_time()`, monotoniczność i rozdzielczość około **41,667 ns**. Oba programy zapisują `time_ns`; jednostka zapisu nie określa dokładności pomiaru. Informacje o zegarach zachowano na stderr (D05-D06).

4. Zmienne i warunki eksperymentu

Czynnik porównywany: konkretna implementacja C++ lub Python. Stałe: maszyna, wspólny plik, kolejność liczb, 200 rund, ręczne pętle, 2 rozgrzewki na proces i 5 prób na blok. Łącznie każda implementacja wykonała **4 rozgrzewki i 10 prób właściwych**. Parametry ustalono przed pomiarami (D01). Rotowana jest kolejność całych bloków, nie pojedynczych prób.

Generator tworzy **20 020** liczb całkowitych z zakresu **0-1000**: każda wartość występuje dokładnie 20 razy. `random.Random(20261017).shuffle` tasuje gotowy multizbiór; **seed = 20261017**. Nie jest to niezależne losowanie każdego elementu. Plik ASCII ma jedną liczbę na wiersz i zakończenia LF. Dane zapisano raz; oba programy odczytują `dane/input.txt`.

Pełny SHA-256 w D03 i D07; poniższe dwa wiersze należy połączyć bez spacji:

```
1e1422a8679f65bb7ec778bdbd821b74a459
6431774711e89671acaa9ddca3d8
```

Weryfikator sprawdził hash bajtów i cały histogram. Hash po serii zgadzał się z hashem generatora przed kompilacją. Polecenia D04 wskazują ten sam plik; kod benchmarków tylko go odczytuje. Odrębnych danych dla języków nie generowano.

5. Kontrola poprawności

Wzorzec niezależny od benchmarków

Dla wartości 0-1000 odległości po obu stronach 500 tworzą dwa ciągi 1-500:

```
1 + ... + 500 = 500 * 501 / 2 = 125250
jedna kopia 0..1000: 2 * 125250 = 250500
20 kopii: 20 * 250500 = 5010000
200 rund: 200 * 5010000 = 1002000000
```

Formułę zapisano przed pomiarami (D01). Po pomiarach D07 dodatkowo policzył wynik innym algorytmem: dla liczb poniżej 500 użył różnicy `500 * count - sum`, a powyżej 500 różnicy `sum - 500 * count`. Użycie bibliotecznej sumy w tej niezależnej kontroli nie zmienia ręcznego algorytmu badanego.

Implementacje i arytmetyka

W obu funkcjach wynik zaczyna od zera. Zewnętrzna pętla wykonuje 200 rund, wewnętrzna przechodzi po wspólnych danych. Oblicza `d = x - 500`, zmienia znak gdy `d < 0`, następnie dodaje d do akumulatora. Nie używa `sum()` ani `abs()` w mierzonej operacji. C++ przechowuje liczby i wynik w `int64_t`, Python w `int`.

Po walidacji wejścia d mieści się w zakresie od -500 do 500, a odległość od 0 do 500. Największy możliwy wynik wynosi `20020 * 500 * 200 = 2002000000`, znacznie mniej niż 9223372036854775807 dla `int64_t`. Iloczyny występują w uzasadnieniu, nie w badanej pętli. Akumulator rośnie monotonnie i również pośrednie wyniki mieszczą się w typie. Python ma arytmetykę całkowitą o dowolnej precyzji.

Poza stoperem każdy wynik jest porównywany ze stałą wzorcową, wypisywany lub zapisywany w diagnostyce rozgrzewki. Błędny checksum przerywa proces kodem 1; błąd wejścia/parametrów kodem 2. Samo sprawdzanie i wypisanie wyniku zapewnia obserwowalność, lecz nie wyklucza wszystkich transformacji kompilatora. Nie użyto `volatile` ani wyłączenia optymalizacji; nie badano asemblera.

Rejestr dowodów i testów

- **D01** - Plan sprzed pierwszego pomiaru; kopia w źródłach D02
- **D02** - Środowisko, SHA źródeł, kopia C++
- **D03** - Metadane generatora, dane
- **D04** - Polecenia, kody, stdout kompilatora, stderr
- **D05** - C++: blok 1 stdout, stderr, blok 2 stdout, stderr
- **D06** - Python: blok 1 stdout, stderr, blok 2 stdout, stderr
- **D07** - Weryfikacja i statystyki, stdout, stderr, kod
- **D08** - Niezależne przeliczenie
- **D09** - Kontrola końcowych artefaktów

TC-01: pełny histogram, count, seed, hash i wzorzec - **zgodne** (D03, D07). TC-02: rzeczywiste flagi Release, źródła, kod kompilatora 0 - **zgodne** (D02, D04). TC-03: nagłówek, typy, dodatnie czasy, numery 1-10, 20 rekordów i 8 rozgrzewek, checksum oraz `true`, wszystkie kody procesów 0 - **zgodne** (D05-D07). TC-04: osobne przeliczenie wzorca, median, rozstępów i wierszy 1 oraz 10 obu implementacji - **zgodne** (D08). TC-05: komplet plików, linki, anonimizacja oraz przegląd renderu eksportu - D09.

6. Procedura

1. Przeczytano materiał i wzorce raportów, sprawdzono Git. Istniejącą zmianę pustego `AGENTS.md` pozostawiono bez modyfikacji. W TiDPI_07 nie było źródeł ani pomiarów; jedynym plikiem był szkolny PDF.
2. Zapisano D01 przed kompilacją i pomiarami; przygotowano obie implementacje.
3. Pierwsze przygotowanie metryczki przerwał sandbox przy `sysctl` (kod 1). Generator, kompilacja i pomiary jeszcze nie ruszyły. Zachowano notatkę przygotowania. Po autoryzacji wykonano serię w nowym katalogu. Notatka jest opisem zdarzenia, nie pełnym logiem ani zanonimizowaną kopią surowych czasów.
4. Zebrano D02, kopie i hashe źródeł; wygenerowano wspólny plik, zapisano D03.
5. Skompilowano C++ w Release (D04). Blok 1: C++, potem Python; blok 2: Python, potem C++. Każdy proces wykonał 2 rozgrzewki, potem 5 prób. Procesy nie działały równocześnie. Zachowano wszystkie czasy.
6. Stoper obejmuje wywołanie funkcji, inicjalizację wyniku, pętle i powrót. C++: `steady_clock::now()` i `duration_cast<nanoseconds>`; Python: `time.perf_counter_ns()`. Odczyt, alokacja i przygotowanie, walidacja wejścia, kontrola wyniku i wypisywanie pozostają poza pomiarem. Zapis czasu zawiera także nieunikniony narzut wywołania zegara.
7. Najpierw sprawdzono wszystkie rekordy i rozgrzewki. Dopiero po akceptacji wyliczono statystyki i złożono pliki `A_*_raw.txt` (D07). D08 sprawdza wybrane rachunki ponownie. Nie wykonano celowego uszkodzenia danych wymaganego w B.

Odtworzenie jednej serii z katalogu zadania:

```
python3 collect_A.py
python3 verify_A.py wyniki/IDENTYFIKATOR_NOWEJ_SERII
```

Identyfikator wypisuje pierwszy skrypt; trzeba zastąpić nim drugą ścieżkę. Każde uruchomienie tworzy nowy katalog wyników. Generator odtwarza wejście z tym samym seedem; wzorcowy hash umożliwia kontrolę zgodności bajtów. Użycie tej samej wersji CPython 3.14.4 usuwa niepewność co do zmiany procedury `shuffle` między wersjami. Można też użyć dołączonego gotowego pliku danych. Weryfikator uruchamiać zwykłym `python3`, bez `-O` (korzysta z `assert`).

7. Surowe wyniki

D05 i D06 to pełne, niezmienione strumienie procesów. Rozgrzewki są osobnymi rekordami na stderr i nie należą do tabeli prób. Poniżej pełna zawartość pochodnych plików `A_cpp_raw.txt` i `A_python_raw.txt`: połączono po dwa bloki, zachowując wszystkie pola i jeden nagłówek. Nie przedstawia się ich jako stdout pojedynczego procesu. Oryginalne bloki pozostają do kontroli.

C++ (D05, agregacja D07):

```
language;trial;time_ns;checksum;correct
cpp;1;808500;1002000000;true
cpp;2;795375;1002000000;true
cpp;3;773084;1002000000;true
cpp;4;840916;1002000000;true
cpp;5;779375;1002000000;true
cpp;6;668208;1002000000;true
cpp;7;730208;1002000000;true
cpp;8;731834;1002000000;true
cpp;9;676125;1002000000;true
cpp;10;659875;1002000000;true
```

Python (D06, agregacja D07):

```
language;trial;time_ns;checksum;correct
python;1;107626500;1002000000;true
python;2;106623500;1002000000;true
python;3;107586333;1002000000;true
python;4;106552750;1002000000;true
python;5;106012416;1002000000;true
python;6;133934750;1002000000;true
python;7;139381458;1002000000;true
python;8;132183500;1002000000;true
python;9;135451250;1002000000;true
python;10;135279125;1002000000;true
```

8. Opracowanie wyników

Tabela 1. Jedna seria A, ta sama konfiguracja, 10 prób na implementację; jednostka **ms**, przeliczenie `time_ns / 1000000`. Źródło: D05-D08.

Miara	C++ Release	Python
Minimum	0,659875	106,012416
Maksimum	0,840916	139,381458
Rozstęp	0,181041	33,369042
Mediana	0,752459	119,905000

Mediana przy 10 próbach to średnia piątej i szóstej wartości po sortowaniu. Dla C++: `(731834 + 773084) / 2 = 752459 ns`. Dla Python: `(107626500 + 132183500) / 2 = 119905000 ns`. Rozstęp to maksimum minus minimum. Nie zaokrąglano danych przed obliczeniem. Minimum pokazuje najszybszą zaobserwowaną próbę, mediana środek rozkładu zachowanych czasów. Żadna z nich nie jest gwarantowanym czasem programu.

D08 ponownie sprawdza te rachunki z użyciem `Decimal`, a także surowe wiersze 1 i 10 obu języków: checksum zgodny z formułą i przeliczenie ns na ms. Przykładowo pierwsza próba C++: `808500 ns = 0,808500 ms`; pierwsza Python: `107626500 ns = 107,626500 ms`. Są to kontrole asystenta; nie deklaruje się ręcznej weryfikacji ucznia.

9. Interpretacja

Obserwacja: w tej serii C++ Release miał niższe minimum i medianę niż ręczna implementacja Python. Obie zwróciły poprawne wyniki we wszystkich próbach oraz rozgrzewkach. Dotyczy to wskazanego pliku, 200 rund i użytych programów.

Obserwacja o kolejności (D08): wszystkie czasy C++ z bloku 2 (0,659875-0,731834 ms) były niższe niż z bloku 1 (0,773084-0,840916 ms). W Pythonie blok 2 (132,183500-139,381458 ms) był wolniejszy niż blok 1 (106,012416-107,626500 ms). Zachowano oba bloki. Zbiorcza mediana Pythona leży pomiędzy grupami, dlatego nie opisuje czasu pojedynczego typowego bloku.

Przyczyny różnic między blokami nie ustalono. Nie mierzono temperatury, częstotliwości, cache ani obciążenia tła. Rotacja kolejności ogranicza stałe uprzywilejowanie miejsca w harmonogramie, ale nie zapewnia identycznych warunków. Sama obserwacja nie pozwala przypisać różnic konkretnemu mechanizmowi sprzętu ani kosztowi interpretera.

10. Ograniczenia eksperymentu

- Jedna maszyna, jedna seria z dwoma blokami, jeden rozmiar i 200 rund. Nie wykonano drugiej pełnej serii B ani badań konfiguracji C-D.
- C++ i Python używają różnych reprezentacji liczb i środowisk wykonania; wspólne jest zadanie i ręczny algorytm, nie instrukcje maszynowe.
- Czas C++ jest krótszy niż 1 ms; narzut i zakłócenia mogą mieć względnie duże znaczenie. Nie kalibrowano rund w osobnym eksperymencie.
- Brak kontroli temperatury, częstotliwości, trybu zasilania i procesów tła. Rozgrzewka nie dowodzi osiągnięcia stabilnego stanu komputera.
- Znana jest zgłoszona rozdzielczość Pythona i nominalny okres C++; nie oszacowano rzeczywistej dokładności ani efektywnej rozdzielczości C++.
- Kompilator może wektoryzować lub inaczej przekształcać pętle. Zgodność checksumu nie dowodzi wykonania każdej instrukcji źródłowej dosłownie.
- Dane są równomiernym multizbiorem po przetasowaniu. Nie reprezentują wszystkich rozkładów ani zastosowań; seed nie jest dowodem jakości losowości.
- Wyniki nie obejmują odczytu, walidacji, wypisywania i startu procesu. macOS/Clang zastępuje szkolny zestaw Windows/MSVC.

11. Wnioski

Poziom A został wykonany w opisanej adaptacji: wspólny plik ma ponad 10 000 liczb, dwie implementacje wykonują uzgodnioną operację, a wszystkie 20 prób i 8 rozgrzewek zgadzają się z niezależnym wzorcem. Release, granice stopera, źródła i pełne strumienie mają dowody D01-D08. Eksport i kontrola końcowa są w D09.

W tej serii mediany wyniosły 0,752459 ms i 119,905000 ms. Wniosek dotyczy tych implementacji i warunków. Różnice między blokami przemawiają za zachowaniem wszystkich prób i ostrożnością w uogólnianiu. Nie deklaruje się wyników B-D ani przewidywanej oceny.

Najważniejsze decyzje do obrony: jeden plik eliminuje różnice generatorów; multizbiór pozwala sprawdzić wynik prostym wzorem; stoper obejmuje tylko pracę obliczeniową; Release pozwala mierzyć zoptymalizowany C++; rozgrzewki są ustalone z góry; checksum kontroluje poprawność, a SHA-256 tożsamość danych.

12. Wykorzystanie AI i sposób weryfikacji

Asystent Codex przeczytał materiał, przygotował plan, kod, zapis dowodów, weryfikator i raport oraz wykonał rzeczywiste uruchomienia. Weryfikacja obejmuje pełny histogram, hash, wzorzec z innej metody, wszystkie rekordy i kody, ponowne rachunki wybranych wierszy oraz przegląd renderu PDF (D07-D09). Nie przypisuje się tych czynności użytkownikowi ani nie deklaruje jego samodzielnej obrony kodu. Przed oddaniem powinien umieć wyjaśnić powyższe decyzje.

Metryczkę zebrano bez nazwy konta i prywatnych ścieżek. Wersja kompilatora zawiera wyłącznie pierwsze dwa wiersze `clang++ --version`, zgodnie z opisem w D02; ścieżki instalacji nie zbierano. Surowych strumieni D03-D06 nie redagowano. Nie commitowano, nie pushowano ani nie wysyłano plików zewnętrznie.